A2 – S4 – 2025/2026 – II.2415
Advanced Algorithms and Programming
Semester Project

LAB_5

Basics of Tree Structures

Session Objectives

By the end of this session, you will be able to:

1. Define and implement binary tree structures for data organization
2. Calculate and analyze tree metrics (height, depth, balance factor)
3. Implement tree traversals (In-order, Pre-order, Post-order) for content processing
4. Apply binary trees to represent hierarchical relationships
5. Convert between tree representations for generalized trees (N-ary trees)

Core Exercises

Exercise 1: Binary Tree Representation of Content Categories
Objective: Implement a binary tree structure to organize social media content categories hierarchically.
Social networks organize content into categories and subcategories (e.g., "Technology" → "Programming" → "Python"). Binary trees provide an efficient way to represent and navigate this hierarchical structure.
Requirements:

1. Create a CategoryNode structure:
    - category_id: unique identifier
    - name: string (category name)
    - post_count: integer (number of posts in this category)
    - left: reference to left child category
    - right: reference to right child category
    - parent: reference to parent node (optional, for navigation)
2. Implement tree metric calculations:
    - calculate_height(node): recursively compute the height of the tree
    - calculate_node_height(node, target_id): find height of a specific category
    - count_nodes(node): total number of categories in the tree
    - count_leaves(node): count categories with no subcategories
    - is_balanced(node): check if tree is balanced (height difference between left and right subtrees ≤ 1 for all nodes)
3. Implement tree property verification:
    - is_full_binary_tree(node)
    - is_perfect_binary_tree(node)
    - is_complete_binary_tree(node)
4. Implement category search and navigation:
    - find_category(category_id, node): search for a category by ID

- find_path_to_root(category_id, node): return list of categories from target to root
- lowest_common_ancestor(id1, id2, node): find deepest common parent of two categories

Examples:

Consider a social network's content category hierarchy:

Level 0: Technology (150)
Level 1: Programming (85)
Level 2: Python (42)
Level 3: Django (18)
Level 3: Flask (12)
Level 2: Java (30)
Level 1: Design (65)
Level 2: UI/UX (38)
Level 2: Graphics (22)

Metric calculations:

Tree height: 3 (from Technology to Django/Flask)
Height of "Java": 2 (Technology → Programming → Java)
Total nodes: 7
Leaf nodes: Django, Flask, Java, UI/UX, Graphics (5 leaves)
Balanced? Yes (left subtree height 3, right subtree height 2 → difference = 1, which is ≤ 1)

Function Examples:

find_category(root, "Python") → returns node with name "Python"
find_path_to_root(root, "Django") → ["Django", "Python", "Programming", "Technology"]
lowest_common_ancestor(root, "Django", "Java") → "Programming"

Complexity Analysis Questions:

1. What is the time complexity of calculating tree height? Prove it.
2. For a perfectly balanced binary tree with n nodes, what is the maximum height?
3. In a social network with millions of categories, why would balance matter?

Exercise 2: Tree Traversals for Content Processing

Objective: Implement the three classic binary tree traversals to process category data in different orders.

Different applications require processing tree data in specific orders:

- In-order: Display categories in sorted order (alphabetical/hierarchical)
- Pre-order: Export tree structure for serialization/replication
- Post-order: Aggregate metrics from children before computing parent totals

Requirements:

1. Create traversal methods for the CategoryNode structure from Exercise 1:
   Part A: In-order Traversal (Left → Root → Right)
   - in_order_collect(node): return list of category names in in-order sequence
   - in_order_accumulate_posts(node): process categories in in-order, building running total of posts

- in_order_find_kth(k, node): find the k-th category in in-order sequence (1-indexed)

Part B: Pre-order Traversal (Root → Left → Right)

- pre_order_export(node): generate tree representation for storage/transmission (formatted string with indentation)
- pre_order_copy(node): create a deep copy of the entire tree
- pre_order_serialize(node): convert tree to string format (e.g., "Technology(150) | Programming(85) | Python(42) | ...")

Part C: Post-order Traversal (Left → Right → Root)

- post_order_total_posts(node): compute total posts in a category including all subcategories
- post_order_average_depth(node): calculate average depth of leaf categories
- post_order_collect_leaves(node): collect all leaf categories

2. Implement traversal-based analytics:
   - find_most_popular_category(node): category with highest post count (considering only the category itself, not children)
   - category_with_most_subcategories(node): category with most direct children

Examples:

Using the category tree from Exercise 1:

In-order traversal output:

Django(18) → Python(42) → Flask(12) → Programming(85) → Java(30) → Technology(150) → UI/UX(38) → Design(65) → Graphics(22)

Pre-order traversal output (formatted export):

```
Technology(150)
  Programming(85)
    Python(42)
      Django(18)
      Flask(12)
    Java(30)
  Design(65)
    UI/UX(38)
    Graphics(22)
```

Post-order total posts calculation:

- For Python node: Django(18) + Flask(12) + Python(42) = 72
- For Programming node: Python subtree(72) + Java(30) + Programming(85) = 187
- For Design node: UI/UX(38) + Graphics(22) + Design(65) = 125
- For Technology node: Programming subtree(187) + Design subtree(125) + Technology(150) = 462 total posts in all categories

Analytics Examples:

distribution_by_depth(root) → {0: 1, 1: 2, 2: 4, 3: 2}

Level 0: 1 node, Level 1: 2 nodes, Level 2: 4 nodes, Level 3: 2 nodes

find_most_popular_category(root) → "Technology" (150 posts)

category_with_most_subcategories(root) → "Programming" (2 children: Python and Java)

Complexity Analysis Questions:

1. What is the time and space complexity of each traversal?

2. When would you use pre-order vs. post-order for aggregating metrics?
3. How would you implement an iterative version of these traversals using a stack?
4. For a social network's category export feature, which traversal is most appropriate and why?

Exercise 3: Generalized Trees (N-ary Trees) and Representations

Objective: Implement generalized trees (N-ary trees) to represent real social network structures where categories can have multiple children, not just two.

Content categories can have many subcategories. Binary trees are a special case; generalized trees (N-ary trees) better represent real-world hierarchies.

Requirements:

1. Create a GeneralizedCategoryNode structure:
    - category_id: unique identifier
    - name: string
    - post_count: integer
    - children: list of child nodes (unlimited)
    - parent: reference to parent (optional)
2. Implement conversion between representations:

    Part A: Binary Tree to Generalized Tree
    - binary_to_generalized(binary_root): convert binary tree to N-ary tree
    - In a binary tree, all nodes (left/right) become children in the generalized tree
    - The hierarchical relationship remains the same, just representation changes

    Part B: Generalized Tree to Binary Tree
    - generalized_to_binary(gen_root): convert N-ary tree to binary tree
    - Use "first child, next sibling" representation
    - Left pointer → first child
    - Right pointer → next sibling
3. Implement generalized tree traversals:
    - pre_order_generalized(node): root, then each child recursively
    - post_order_generalized(node): all children, then root
    - level_order_generalized(node): breadth-first traversal using a queue
    - calculate_fan_out(node): maximum number of children any node has
4. Implement tree metrics for generalized trees:
    - calculate_height_generalized(node): longest path from root to leaf
    - count_nodes_generalized(node): total nodes
    - count_leaves_generalized(node): nodes with no children
    - calculate_branching_factor(node): average number of children per non-leaf node

Examples:

Conversion Example:

```
# Binary tree node (before conversion):
Node "Programming":
  left = Python (first child in generalized)
  right = Design (next sibling in generalized)
# After binary_to_generalized:
Generalized Node "Programming":
```

children = [Python, Design] # Both become children

Traversal Examples:

- pre_order_generalized(root) → Technology, Programming, Python, Django, Flask, Java, Design, UI/UX, Graphics, Business, Finance, Marketing, HR
- level_order_generalized(root) → Technology, Programming, Design, Business, Python, Java, UI/UX, Graphics, Finance, Marketing, HR, Django, Flask
- calculate_fan_out(root) → 3 (Technology has 3 children: Programming, Design, Business)
- calculate_branching_factor(root) → (3 + 2 + 2 + 3) / 4 = 2.5 average children per non-leaf

Complexity Analysis Questions:

1. What are the advantages and disadvantages of each representation (binary vs. generalized)?
2. For a generalized tree with n nodes and branching factor b, what is the space complexity of each representation?
3. How does traversal complexity differ between binary and generalized representations?
4. In a social network with millions of categories, which representation would you choose and why?
5. Explain the "first child, next sibling" transformation and its time complexity.

Final Integration Question

Feature Design: "Content Category Analytics Dashboard"

Design a comprehensive analytics dashboard for content managers that combines all tree concepts:

Components:

1. Binary Tree Category Hierarchy (Exercises 1-2): Organize content categories with metrics
2. Generalized Tree Representation (Exercise 3): Handle real-world multi-category relationships where categories can have multiple subcategories

Requirements:

- Content managers can view the complete category hierarchy
- System displays metrics for each category (post count, depth, number of subcategories)
- Managers can export the category structure for backup/sharing
- System can identify unbalanced parts of the hierarchy that need restructuring
- System can find the most popular categories (by post count)
- Analytics show distribution of categories by depth level

Describe:

1. How the binary tree and generalized tree representations work together in your design
2. Which traversals would be used for which features (export vs. metrics calculation vs. searching)
3. How you'd handle deep hierarchies (1000+ levels) without stack overflow
4. Performance considerations for:
    - Calculating total posts across all categories (which traversal?)
    - Finding all leaf categories (categories with no subcategories)
    - Generating a formatted report of the entire hierarchy

5. Trade-offs between using binary tree representation vs. generalized representation for different operations

Example Scenario:

A content manager wants to:

1. View the entire category tree structure
2. Find which category has the most subcategories
3. Calculate the total number of posts in the "Technology" branch
4. Export the tree structure to share with the team
5. Identify if any branch is too deep (unbalanced)

Which traversals and representations would you use for each task?

General Instructions

Repository Setup
Create branch: LAB _05_BasicsOfTrees

File Structure:
Follow the same structure of LAB _04

Git Requirements:
Same as for LAB _04

Documentation & Notation

At the End of the Session	Before Friday Midnight of the Same Week
Note: 10	**Note: 10**
• Each Team (of number BB) Upload at the **end of the session** a very short PDF document and name it **TBB_LAB5_AES.pdf**, containing for each exercise: the algorithm in Pseudo-Code (according to the rules explained in lecture 1) – Brief answers of the questions, • You can implement your algorithms to make sure that are working, • NO AI-generated content (will be checked for authenticity), • NO code snippets.	• You finish all the implementation of all exercises, • You finalize the previous report and give it a new name **TBB_LAB5_BFM.pdf** and upload it before **Friday midnight of the same week**, containing for each exercise: same as previous + more complexity analysis and reflection + Test set for edge cases of your Algorithms' implementation, • AI-generated content is not recommended, • A link to the Team's Repository in GitHub.